Performance / Load Test Report and Optimization Results

System under test: Campus Ride-Pooling and Split-Fare System
Experiment script: `performance/k6/ride-search.js`
Core implementation: `apps/web/src/lib/ride-search.ts`
Test date: **2026-10-29**
Environment/commit: `8120a92` (optimized branch snapshot)

The application is a Next.js App Router service backed by PostgreSQL. Authenticated ride-search requests render `/rides/results`; the server validates the session and URL filters, executes the PostgreSQL search, fetches occupant display names in one batch query, and renders the result page. Ride creation and join-request acceptance are separate transactional workflows using `pg` transactions and row locks. The experiment below targets ride discovery because it is a high-frequency read path and directly exercises the optimized database query.

1. Define measurable performance targets

The rubric workload is modeled as **100 concurrent daily active users** and **300 completed ride-search transactions**: 100 virtual users, each performing three iterations. This exceeds the required 250+ core transactions while keeping the experiment deterministic.

Metric	Target	Measurement
Successful transactions	>= 99% HTTP 200	k6 `checks` and `http_req_failed`
p50 response latency	<= 300 ms	k6 `http_req_duration`
p95 response latency	<= 750 ms	k6 threshold
p99 response latency	<= 1,500 ms	k6 threshold
Throughput at 100 VUs	>= 25 transactions/second	k6 `http_reqs / test duration`
Correctness	100% valid HTML responses	k6 status check
Ramp acceptance	No more than 1% failed requests through 100 VUs; record the first load level where p95 exceeds 2 seconds	k6 `ramp` mode
Stress behavior	At 400 VUs, the service should remain observable, return >= 90% valid responses, and fail without data corruption	k6 `stress` mode

These are application-level targets measured from the load generator to the web server, including application code and database time. They are not claims about network latency outside the test environment.

The targets deliberately distinguish normal-load acceptance from stress behavior. The system is expected to degrade under overload; the stress test measures where that happens and whether degradation is controlled rather than treating overload as an ordinary success case.

2. Conduct a reproducible load/performance experiment

Preconditions

1. Start PostgreSQL and the web application using the repository setup instructions.
2. Run the schema migration so the `riders_active_ride_idx` index exists.
3. Prepare 100 valid, verified rider session-cookie values. Store them in a local file or environment variable, never in this repository or shell history. The script accepts `SESSIONS_FILE`, `SESSIONS_JSON`, or one cookie in `SESSION_COOKIE` for a smaller dry run.
4. Ensure the database contains active rides and the configured IDs exist. Defaults are `kanpur-central` and `hall-6`; override them with `HUB_ID` and `CAMPUS_LOCATION_ID`.

Baseline run

Run the baseline revision before applying the SQL optimization:

```
$env:BASE_URL = "http://localhost:3000"
$env:PERF_PHASE = "baseline"
$sessionFile = "C:\path\outside\repository\sessions.json"
docker run --rm --add-host=host.docker.internal:host-gateway `
  -v "${PWD}:/work" -v "${sessionFile}:/work/sessions.json:ro" -w /work grafana/k6 run `
  -e BASE_URL=http://host.docker.internal:3000 `
  -e PERF_PHASE=baseline `
  -e SESSIONS_FILE=/work/sessions.json `
  performance/k6/ride-search.js
```

Measured baseline run on the local Docker stack:

- 300/300 completed transactions; 100% checks passed; 0% HTTP failures.
- Throughput: **not retained in the raw k6 summary**; the earlier 106.72 value was an `http_reqs`-style rate and is not used as transaction throughput.
- p50: **303.23 ms**; p95: **2.27 s**; p99: **2.57 s**.
- Baseline revision: `69aead7`.

Optimized run

Apply the optimization in the current revision, restart the application, and repeat the same command with `PERF_PHASE=optimized`:

```
$env:PERF_PHASE = "optimized"
docker run --rm --add-host=host.docker.internal:host-gateway `
  -v "${PWD}:/work" -v "${sessionFile}:/work/sessions.json:ro" -w /work grafana/k6 run `
  -e BASE_URL=http://host.docker.internal:3000 `
  -e PERF_PHASE=optimized `
  -e SESSIONS_FILE=/work/sessions.json `
  performance/k6/ride-search.js
```

Measured optimized run on the same local Docker stack:

- 300/300 completed transactions; 100% checks passed; 0% HTTP failures.
- Throughput: **not retained in the raw k6 summary**; the earlier 137.53 value was an `http_reqs`-style rate and is not used as transaction throughput.
- p50: **289.84 ms**; p95: **1.76 s**; p99: **2.00 s**.

- The p95 and p99 thresholds were not met, but both improved substantially.
- Optimized revision: `8120a92`.

The same database shape, session set, machine, k6 version, and query parameters were used for both runs.

The script uses a `per-vu-iterations` scenario with `vus: 100` and `iterations: 3`, so the rubric workload is exactly 300 search transactions. It also provides release-testing profiles aligned with the supplied guidance:

Stepped-load run

This steadily increases load through 25, 50, 100, 150, 200, and 250 VUs. Record the first stage where p95 exceeds 2 seconds or the failed-request rate exceeds 1%.

```
docker run --rm --add-host=host.docker.internal:host-gateway `
  -v "${PWD}:/work" -v "${sessionFile}:/work/sessions.json:ro" -w /work grafana/k6 run `
  -e BASE_URL=http://host.docker.internal:3000 -e TEST_MODE=ramp `
  -e SESSIONS_FILE=/work/sessions.json performance/k6/ride-search.js
```

Measured stepped-load result: 9,991 successful ride-search transactions over the two-minute ramp, with 0.00% HTTP failures and 100.00% successful checks. Aggregate throughput was **82.49 requests/second**; median latency was **205.34 ms**, p95 was **933.16 ms**, p99 was **not emitted by k6 in this run**, and the maximum observed latency was **1.22 seconds**. The aggregate p95 exceeded the normal-load target of 750 ms, while the 2-second early-warning criterion was not exceeded. Because the current ramp script reports one aggregate percentile rather than per-stage percentiles, the exact first unacceptable VU stage cannot be isolated from this run; the report records the conservative conclusion that the aggregate ramp did not meet the 750-ms release target.

Stress run

This deliberately overloads the service through 100, 200, 300, and 400 VUs. The test records response failure, latency degradation, and whether the service remains healthy.

```
docker run --rm --add-host=host.docker.internal:host-gateway `
  -v "${PWD}:/work" -v "${sessionFile}:/work/sessions.json:ro" -w /work grafana/k6 run `
  -e BASE_URL=http://host.docker.internal:3000 -e TEST_MODE=stress `
  -e SESSIONS_FILE=/work/sessions.json performance/k6/ride-search.js
```

Measured stress result: 15,263 successful ride-search transactions over the 80-second overload profile, reaching **400 VUs**, with **0.00% HTTP failures** and **100.00% successful checks**. Throughput was **187.53 requests/second**; median latency was **985.86 ms**, p95 was **1.82 seconds**, p90 was **1.57 seconds**, and the maximum was **6.91 seconds**. No request-failure point was reached, although tail latency degraded substantially under overload. After the run, `GET /api/health` returned **HTTP 200** with `{"status":"ok"}`, demonstrating service recovery and continued observability.

A run is reproducible with:

- k6 version: **2.3.0 (grafana/k6:latest)**
- Node/application version: **24.x / Next.js 16.3.8**
- PostgreSQL version: **16**
- application URL: `http://localhost:3000`
- dataset size: **100 authenticated sessions, 300 scheduled rides, 300 active owner seats**

3. Measure response latency and throughput

The retained raw counters are 300 iterations, 300 HTTP requests, 300 successful checks, and 0 failed HTTP requests for each rubric run. Total duration and `iteration_duration` were not retained, so valid transaction throughput cannot be reconstructed without rerunning k6.

Result	Baseline	Optimized	Change
Transactions completed	300	300	0% change
Throughput (transactions/s)	Not retained	Not retained	Not recomputable
p50 latency	303.23 ms	289.84 ms	**4.4% decrease**
p95 latency	2.27 s	1.76 s	**22.5% decrease**
p99 latency	2.57 s	2.00 s	**22.2% decrease**
HTTP failure rate	0%	0%	no regression

Throughput must be calculated as completed iterations divided by total elapsed test time. Latency is the k6 `http_req_duration` value, not a client-side timestamp estimated from logs. Because the script sleeps for one second per iteration, 300 rubric iterations cannot complete in less than approximately three seconds; the old rates were invalid as transaction throughput.

4. Identify at least one performance bottleneck

The bottleneck was repeated active-occupant counting in the ride-search SQL. For every candidate ride, the old query scanned `riders` separately to calculate `occupant_count`, `seats_left`, the full-ride predicate, and the optional fare-share predicate. With `R` candidate rides and `S` active seat rows, this creates repeated correlated work and increases database CPU/IO as the ride and seat tables grow.

Before: inefficient query pattern

```
(SELECT COUNT(*)::int FROM riders o
 WHERE o.ride_id = r.id AND o.left_at IS NULL) AS occupant_count,

GREATEST(
  r.capacity_snapshot -
  (SELECT COUNT(*)::int FROM riders o
   WHERE o.ride_id = r.id AND o.left_at IS NULL),
  0
) AS seats_left

-- The same COUNT(*) was repeated again in the available-seat and fare filters.
```

The code also had no partial index dedicated to active seats. The existing unique `(ride_id, user_id)` index is not equivalent to an index that excludes departed riders (`left_at IS NOT NULL`).

Broader performance review

The full source review also examined authentication/session validation, ride creation, join requests, trip/settlement, notifications, chat, scheduled jobs, and admin queries. The following issues were confirmed and addressed or recorded:

Area	Finding	Action
Auth/session	Every authenticated request previously updated `sessions.last_activity_at`, creating write/WAL and row-lock pressure.	Refresh only when the activity timestamp is older than five minutes.
Payment reminders	Up to 500 rows were updated and notified with two awaited statements per row.	Use one set-based `UPDATE ... RETURNING` plus notification insert.
Ride membership	Active membership checks were not supported by a user-leading partial index.	Add `riders_active_user_ride_idx`.
Search	Open ride filtering lacked a search-oriented partial index.	Add `rides_open_search_idx`.
Notifications	Cursor ordering used `(created_at, id)` but the index omitted `id`.	Extend `notifications_user_created_idx`.
Complaint AI queue	Latest-analysis lookup ordered by timestamp and ID.	Extend the complaint-analysis index with `id`.
Chat/request history	Chat messages and request history are unbounded.	Record as follow-up pagination work; no response contract was silently changed.
Notification offset pagination	Deep offsets require PostgreSQL to walk and discard older rows.	Record cursor pagination as follow-up work; existing clients still depend on offset fields.

5. Perform at least one optimization

The optimization has two parts:

1. ride-search.ts now uses one `LEFT JOIN LATERAL` aggregate to compute `occupant_count` once per candidate ride. The projection, available-seat predicate, and fare-share predicate all reuse that value. This preserves the existing response shape and filtering semantics while removing repeated correlated counts.
2. schema.sql adds `riders_active_ride_idx ON riders(ride_id) WHERE left_at IS NULL`, allowing the aggregate to scan only active seats using the join key.

After: optimized query pattern

```
LEFT JOIN LATERAL (
  SELECT COUNT(*)::int AS occupant_count
  FROM riders active_rider
  WHERE active_rider.ride_id = r.id
    AND active_rider.left_at IS NULL
) occupant_counts ON true

-- Reused everywhere:
occupant_counts.occupant_count
r.capacity_snapshot - occupant_counts.occupant_count
CEIL(r.expected_total_fare::numeric / (occupant_counts.occupant_count + 1))
```

This is a database-side optimization rather than an application cache: it avoids stale availability/fare data and retains transactional correctness. The index migration must be applied before the optimized run.

6. Compare measurements before and after optimization

The optimization is successful if the optimized run meets the targets in section 1 and improves the database-bound latency/throughput relative to the baseline under the same workload. Fill in the measured comparison below:

Comparison	Result
p95 latency improvement	**22.5%**
p99 latency improvement	**22.2%**
Throughput improvement	Not recomputable from retained output
Target status	**Reliability and correctness passed; p95 and p99 did not meet target, and throughput requires a rerun with retained duration data**
Bottleneck evidence (`EXPLAIN (ANALYZE, BUFFERS)`)	No plan output was retained; rerun the documented query with `EXPLAIN (ANALYZE, BUFFERS)` before claiming plan-level timing evidence.

Interpretation: a lower p95/p99 and higher transactions-per-second indicate that the database search path is doing less repeated work. If the results do not improve, record that outcome rather than changing the numbers; inspect the PostgreSQL execution plan, connection-pool saturation, and dataset size before drawing a conclusion. The retained evidence contains one run per phase, not the two runs required for a median. Repeat each phase before claiming a median:

- Baseline run 1: retained latency values above; duration not retained.
- Baseline run 2: not run.
- Optimized run 1: retained latency values above; duration not retained.
- Optimized run 2: not run.

The completed run demonstrates lower recorded p50/p95/p99 latency, but it does not satisfy the p95/p99 targets and does not provide valid retained throughput or a two-run median. Rerun the rubric twice per phase with raw k6 summaries and capture the requested `EXPLAIN (ANALYZE, BUFFERS)` output before making a stronger performance conclusion.